

팀프로젝트

유기동물 보호소 관제 시스템

작성자: 서예현

1차 팀 프로젝트

사용한 도구

사전 조사

구현 된 화면들

아이디어 구현화

데이터베이스

1. 데이터베이스 작성 및 시각화

2. 데이터베이스 연동 및 관리

실시간 영상 관제

1. ESP32-CAM 송출 오류

경보 알림 기능

1. 메일 경보 발송 이메일 연동

2. 디스코드 웹후크 경보 발송

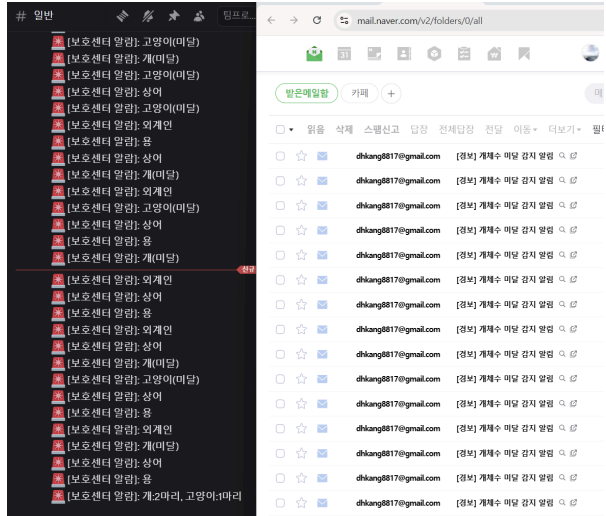
관제 화면 UI/UX 편의성 강화

이후의 개선점

프로젝트를 마치며

1차 팀 프로젝트

수강 중인 KDT의 팀프로젝트가 시작되었다. 주제는 유기동물 보호소의 보호 동물 개체수 파악 및 외부 야생동물 감지, 이와 연동한 실시간 알림 시스템이다. 보호소 내에서 보호 중인 동물의 마릿수를 업데이트 하면, 실시간으로 해당 내용이 기준치가 되는 디렉터리에 업데이트 된다. 업데이트 한 영상이나 esp가 송신하는 화면 내에서 기준치에 미달하는 내역이 감지되는 경우, 해당 내역을 데이터베이스에 저장 하고, 경보 알람을 발송한다. 이와 더불어 위험 객체로 지정한 야생 동물이 감지되는 경우에도 마찬가지로 경보 알람이 발생하는 방식이다.



[그림 1] 알람 발송 시스템

사용한 도구

하드웨어: ESP32-cam, Pydrone

언어: Python

웹 스트리밍 서버: Flask

영상 감지: YOLOv8n

데이터베이스: SQLite

개발 환경: VScode, Arduino, Thonny

사전조사

해당 프로젝트를 시작한 후, 조사를 위해 실제 사설 보호소에 봉사 활동을 다녀오게 되었다. 해당 보호소에서 들은 이야기는 사설 보호소의 열악한 환경을 여과없이 보여주고 있었다.



[그림 2] 실제 방문한 보호소 사진

해당 보호소에는 500마리를 넘는 유기견이 보호 되고 있었고, 고작해야 신선하고 시원한 물과 사료들을 챙겨주고, 견사 내부를 청소하는 것 만으로도 하루가 부족할 지경이었다. 보호소 봉사 활동은 대체로 공공 보호소에 집중 되곤 한다. 사설 보호소는 열악한 시설에 더불어, 만성적인 인원 부족을 겪고 있었다. 하물며 소장님은 24시간 보호소에 상주하시며 밤낮의 구분 없이 크고작은 문제 상황에 대비하신다 말씀 하셨다.

이 이야기를 듣고, 현재 진행 중인 프로젝트를 통하여 이러한 열악한 환경의 보호소에서 근무하는 직원들이 과중한 업무의 중간에 잠깐이라도 휴식을 취할 수 있기를 바르게 되었다.

벤치마킹

해당 프로젝트를 구현하기 위해 참고 한 기술은 크게 세 가지이다.

1. 극지 연구를 진행하는

월동연구대의 **펭귄 개체수**

파악 기술 (그림 3의 좌상단)

월동연구대에서는 2019년도, 펭귄의 개체수 파악을 위해 새로운 기술을 도입하였다. 드론을 이용하여 펭귄의 서식지를 촬영한 후, 해당 영상과 AI를 이용하여 펭귄의 개체수를 파악하는 것이다. 현재로서는 정확성이



[그림 3] 벤치마킹 기술들
(좌 상단: 월동연구대의 드론이 촬영한 펭귄 사진,
좌 하단: 출현 광고의 실시간 알림 화면,
우 상단: 로드킬 방지 시스템의 가동 모습)

떨어지며, 객체 겹침 현상을 확실하게 해결하지 못하여 사람이 2차적인 파악을 실시하고 있으나, 사람이 손을 대지 않고 펭귄의 개체수와 서식지, 둥지 등을 비교적 정확하게 파악 할 수 있다는 것은 자명하다. 해당 기술을 참고하여 프로젝트의 **보호 동물 개체수 감지 시스템**을 구축 할 수 있었다.

2. 경기도 양평군, 강원도 평창군 등에 설치 된 **로드킬 방지 스마트 시스템**. (그림 3의 우측)

로드킬 방지 스마트 시스템은 이름 그대로, 국도 상에서 로드킬을 방지하는 시스템이다. 인공지능 기술 기반의 CCTV 및 레이더 센서를 설치하여 동물 출현을 감지 하고, 일정 거리

밖에 설치 한 LED 전광판으로 야생 동물이 감지되면 메시지를 출력한다. 해당 기술을 프로젝트의 **야생 동물 감지**에 참고 할 수 있었다.

3. 홈캠의 실시간 **모바일 푸시 알림** (그림 3의 좌하단)

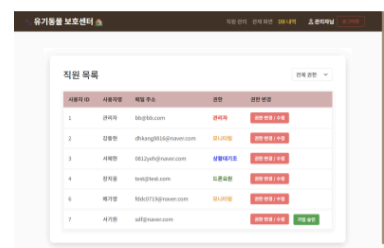
반려 동물을 키우는 보호자나, 아이를 키우는 부모의 경우, 집에 홈캠을 설치하는 경우가 잦다. 홈캠은 지정된 범위를 촬영하다가, 움직임이 발생하는 등의 조건이 만족되면 사용자의 스마트폰으로 푸시 알람을 보낸다. 이러한 기능을 참고하여 프로젝트의 **실시간 알림 시스템**의 방향을 잡을 수 있었다.

해당 레퍼런스를 활용하여, 개체수 감지(월동연구대), 객체 탐지(로드킬 방지), 실시간 알람(홈캠) 시스템을 성공적으로 구축 할 수 있었다.

구현 된 화면들

시나리오를 구현하기 위해 정의 된 화면들은 크게 세 가지이다.

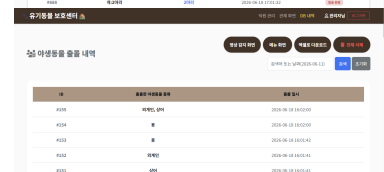
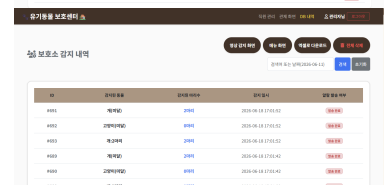
1. 신규 등록 요청자의 가입을 승인하고, 해당 시스템을 이용하는 직원의 정보를 변경하는 직원 관리 페이지. (그림 4)
2. ESP32 등을 이용한 실시간 영상을 송출하고, YOLO를 이용하여 분석 및 감지를 실시하는 관제 페이지. (그림 5)
3. YOLO가 감지 한 내역이 저장된 데이터베이스 및, 감지 기준치의 변경이 가능한 데이터베이스 관리 페이지. (그림 6)



[그림 4] 직원 관리 페이지



[그림 5] 영상 관제 페이지



[그림 6] 시각화 한 데이터베이스

아이디어 구현화

레퍼런스를 모으고, 프로젝트의 계획을 세운 후, 본격적인 개발에 들어갈 수 있었다. 그 중 데이터베이스 작성 및 백엔드 구축을 주도적으로 진행하였다.

데이터베이스

1. 데이터베이스 작성 및 시각화

시작할 때 필요한 데이터베이스 테이블은 총 4개였다.

1. 사용자(Users) 데이터베이스
2. 유기동물 개체수 감지 내역(Animals) 데이터베이스
3. 이상객체 감지 내역(Dangers) 데이터베이스
4. 경보 발송(Alarms) 데이터베이스

이 중 경보 발생의 경우, 이상객체 감지 내역

데이터베이스에서 고유번호와 감지 일시를, 사용자

데이터베이스에서 경보 발송 대상의 E-mail을 참조하여 가져올 예정이었다.

그러나, 개발을 진행하던 중, 현재 보호 중인 유기 동물의 개체수, 즉 기준치가 될 내용을 저장 해 둘 테이블의

필요성을 느꼈다. 또한 경보 발송 테이블을 별도로 정의하는 것 보다는 감지 된 내용에서 곧장 알람 발송 여부를 판단하는 것이 더욱 효율적일 것이라 판단하게 되었다.

최종적으로 제작 된 데이터베이스 테이블은 여전히 4개이지만, 사용자(Users), 현 시점에서 보호 중인 유기 동물(Animals), 개체수 감지 내역(Detect_animal), 위험객체 감지

내역(Dangers)로 구성되었다. 이 중 위험객체의 경우 감지 되어 데이터베이스에 저장 되는 것이 경보 발생의 조건이었으나, 개체수 감지의 경우 특정한 조건을 만족했을 경우에만 경보가 발생 하는 시스템으로 구성되어 있었다. 때문에 어떤 상황에서 경보가 발생하였는지 시각적으로 확인 할 필요가 있었고, 이를

데이터베이스			
시스템명	Stray Animal Protection System	작성일자	2026-05-19
주제명/영역		데이터베이스명	
데이터베이스	users	데이터베이스	
데이터베이스	사용자 DB	데이터베이스	
번호	필요성	속성명	도메인
1	user_id	사용자 고유번호	N/A
2	user_name	사용자 이름	N/A
3	user_email	사용자 이메일	N/A
4	user_role	사용자 권한	N/A

데이터베이스			
시스템명	Stray Animal Protection System	작성일자	2026-05-19
주제명/영역		데이터베이스명	
데이터베이스	animals	데이터베이스	
데이터베이스	유기동물 DB	데이터베이스	
번호	필요성	속성명	도메인
1	animal_id	유기동물 고유번호	N/A
2	animal_spec	유기동물 종류	N/A
3	animal_count	유기동물 감지 마리수	N/A
4	animal_detect	유기동물 감지 일시	N/A

데이터베이스			
시스템명	Stray Animal Protection System	작성일자	2026-05-19
주제명/영역		데이터베이스명	
데이터베이스	dangers	데이터베이스	
데이터베이스	이상 객체 DB	데이터베이스	
번호	필요성	속성명	도메인
1	danger_id	이상 객체 고유번호	N/A
2	danger_spec	이상 객체 종류	N/A
3	danger_detect	이상 객체 감지 일시	N/A

데이터베이스			
시스템명	Stray Animal Protection System	작성일자	2026-05-19
주제명/영역		데이터베이스명	
데이터베이스	alarms	데이터베이스	
데이터베이스	알람 DB	데이터베이스	
번호	필요성	속성명	도메인
1	alarm_id	알람 고유번호	N/A
2	danger_id	이상 객체 고유번호 참조	N/A
3	danger_detect	이상 객체 감지 일시 참조	N/A
4	alarm_email	알람 발송 대상 이메일	N/A

[그림 7] 실제로 작성한 테이블 정의서

TABLES	
>	alembic_version
>	animals
>	dangers
>	detect_animal
>	users

[그림 8] 최종 확정 된 DB

발송 완료
미발송
발송 완료

[그림 9] 불린값의 시각화

해결하기 위해 경보 발생 여부를 확인 할 수 있는 detect_alarm 컬럼을 추가하여 불린값을 부여하였다. 이를 통해 시각화 대시보드에서는 '알람 발송 여부' 컬럼을 '미발송' 혹은 '발송 완료' 상태로 확인 할 수 있게 되었다.

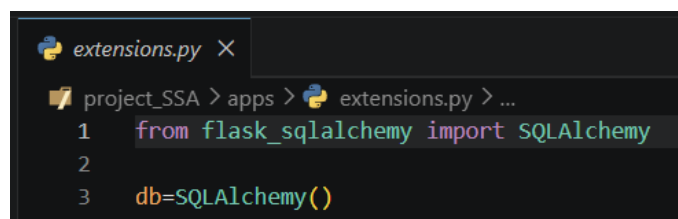
2. 데이터베이스 연동 및 관리

테이블 작성을 마친 이후, YOLO가 감지 한 내역을 데이터베이스에 저장하거나, 시각화 대시보드에서 변경 한 내용이 즉각 반영 되도록 연동을 실시하였다. 해당 과정에서 YOLO의 감지 내역이나, 기준치가 되어줄 딕셔너리가 DB와 연동 되지 않는 문제가 발생하였다. 이는 YOLO의 감지 내역을 넘겨주는 Tensor의 값과 DB의 테이블에서 받을 수 있는 값이 일치하지 않아 생긴 문제였다. 이에 더해 YOLO는 리스트 구조로 데이터를 넘겨주었으나, DB에 저장 될 때는 이 리스트를 언패킹 해줘야 한다는 부분을 간과하여 발생한 문제였다.

해당 문제를 해결하기 위해 YOLO가 감지 한 결과값의 내역을 직접적으로 추출하고, SQLite 자체에 입력 함으로써 해결 할 수 있었다. 기준치 딕셔너리 또한 비슷한 원인으로, 해당 문제는 DB가 딕셔너리에서 받을 수 없는 값을 반환하였기 때문이었다. 이 또한 SQLite를 직접적으로 호출하여 컬럼 내부 값을 직접 딕셔너리에 입력하는 방식으로 전환하며 해결 할 수 있었다.

또한 이 모든 과정을 거친 후, DB만을 담당하는 extensions.py 파일을 만들어 순환 참조를 방지하였다.

이후로는 DB 시각화 대시보드에 출력 되는 내용이 한글이 되도록 딕셔너리를 추가하였다.



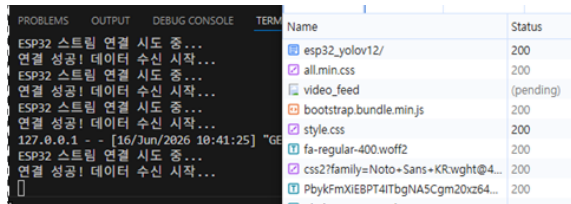
[그림 10] 순환참조 방지를 위한 extensions.py

ID	감지된 동물	감지된 마리수	감지 일시	알람 발송 여부
#691	개(미달)	2마리	2026-06-18 17:01:52	발송 완료
#692	고양이(미달)	0마리	2026-06-18 17:01:52	발송 완료
#693	개:2마리	2마리	2026-06-18 17:01:52	발송 완료

ID	출몰한 야생동물 종류	출몰 일시
#155	외계인, 상어	2026-06-18 16:02:00
#154	용	2026-06-18 16:02:00

[그림 11] 감지 된 내역이 한글로 출력 되는 DB 시각화 대시보드

실시간 영상 관제



Name	Status
esp32_yolov12/	200
all.min.css	200
video_feed	(pending)
bootstrap.bundle.min.js	200
style.css	200
fa-regular-400.woff2	200
css2?family=Noto+Sans+KR:wght@4...	200
PlykFmXIE8PT4ITbgNA5Cgm20x64...	200

[그림 12] 당시 발생하였던 ESP 송신 오류 중 하나. ESP32 연결은 성공적이거나, Flask 서버에는 pending 상태가 지속되었다.

1. ESP32-CAM 송출 오류

프로젝트의 시작부터 마지막까지 ESP32는 꾸준한 문제를 일으켰다. 영상이 정상적으로 송출 되지 않는 탓에 팀원 전원의 골머리를 앓게 만들었다.

해당 문제는 ESP32 카메라가 송신한 데이터를 Flask 서버가 받지 못하여 생긴 문제였다. 브라우저의 개발자 도구 내의 네트워크 탭은 새로고침을 할 때 마다 video_feed, pending이라는 단어만 보여주었다. 이 문제를 해결하기 위해 시도해본 방법은 총 세가지였다.

1. requests 방식

해당 문제를 해결하기 위해 opencv가 아닌 리퀘스트 방식을 이용해보았으나, JPEG segment 문제를 뱉어내며 작동하지 않았다.

2. Flask 비경유 방식

이후 다양한 방법을 시도하던 중, Flask를 경유하지 않고 직접적으로 ESP32 카메라 화면을 띄우는 데 성공하였으나, 이 방법은 YOLO의 객체 감지를 실시할 수 없었다. 프로젝트의 목표는 YOLO를 이용한 객체감지였기 때문에, 해당 방법 또한 폐기해야 했다.

3. 멀티 스레딩

결국 답을 찾은 것은 Threading 방식이었다. 원래는 하나의 함수에서 실행 하던 영상 수신, YOLO 감지, 영상 송출의 과정을 두개의 함수로 나누었다. 첫 번째 함수는 영상을 받아, 각 프레임별로 YOLO 감지를 진행한다. 이후 해당 프레임을 current_frame이라는 변수에 저장한다. 그러면 두 번째 함수가 current_frame에 저장되어있는 각 프레임을 읽어들이, 사용자의 화면에 송출한다.

해당 방법을 통하여 ESP32 카메라 화면이 송출 됨은 물론, YOLO를 이용한 객체 감지 또한 정상적으로 작동하였다.

결과적으로 모든 작업을 하나의 함수가 수행하고 있기 때문에 발생한 문제로, 멀티스레딩 환경을 구축하여 입출력을 구분함으로서 해결 할 수 있었다.

경보 알림 기능

☆ [경보] 개체수 미달 감지 알림 

보낸사람 dhkang8817@gmail.com

2026년 6월 18일 (목) 오후 6:22

관리자님,

현재 개체수 미달이 감지되었습니다. dog
시스템을 확인해 주세요.

[그림 13] 실제 발송 된 경보 알림

```
email_thread = threading.Thread(target=send_alert_email, args=(animal,), daemon=True)
email_thread.start()

discord_thread = threading.Thread(target=send_discord_webhook, args=(type_string,), daemon=True)
discord_thread.start()
```

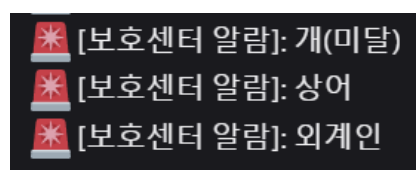
[그림 14] 경보 발송 로직. 스레딩을 이용해 비동기처리 한다.

직원만 받는 시나리오였기 때문에, filter_by를 이용하여 상황대기조의 테이블만 추출 한 후 E-mail을 참조하였다.

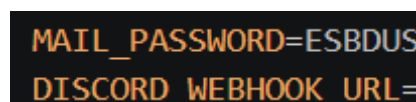
메일 발송의 경우, 스레딩 방식을 이용하여 비동기 처리 할 수 있도록 코드를 작성하였다.

1. 메일 경보 발송 이메일 연동

메일 발송의 경우, 팀원이 구현 한 코드에 DB를 연동 시키는 것으로 마무리 될 수 있었다. 감지 내역과 연동 시켰을 때처럼, SQLite 자체를 읽어들여 그 안에서 조건에 맞는 E-mail만 참조 하는 방식을 이용했다. 경보 알림의 경우, 상황대기조로 구분 된



[그림 15] 실제 발송 된 디스코드 경보



[그림 16] .env 파일로 비밀번호와 url을 분리하여 보안성을 강화하였다.

2. 디스코드 웹후크 경보 발송

본래는 조금 더 접근이 용이한 메신저를 이용하고자 하였다. 그러나 서비스 비용 정책 및 API 접근성을 고려하여, 개발 자유도가 높은 디스코드를 최종적으로 선정하였다. 디스코드는 실시간 웹후크를 지원하고 있어 경보 시스템을 구현화 하기 알맞은 플랫폼이었다. 디스코드를 이용한 경보 발송 또한 스레딩 방식을 이용하여 비동기 처리 하였다.

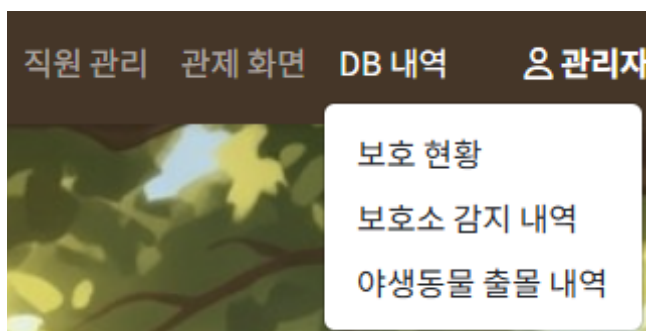
경보 알람 발송의 경우, 그림 16의 내용처럼 .env 파일에 메일 비밀번호와 웹훅 URL을 분리했다. 이를 통하여 보안을 강화할 수 있었다.

관제 화면 UI/UX 편의성 강화

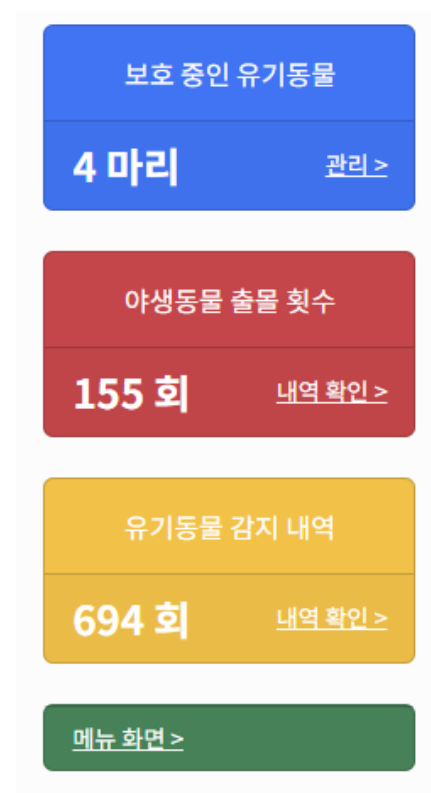
개발 초기의 관제 화면은 YOLO가 감지 중인 화면과, 실시간 감지 로그만 업로드 되는 다소 밋밋한 화면이었다. 이후 감지 화면의 상단으로 몇 가지 버튼이 생겼으나, 디자인 적으로는 다소 엉망이었다. 해당 프로젝트를 완료하기 직전에, Weweb 등의 무료 bootstrap 사이트를 통하여 몇가지 원하는 버튼과 디자인을 찾아낼 수 있었다.

관제 화면에 각 DB 시각화 대시보드로 라우트 될 수 있는 버튼을 추가하여 관제 화면의 편의성을 증대하였다. 이후 헤더에도 각 페이지로 라우트 되는 버튼을 추가하여

페이지
간



[그림 18] 각 페이지의 접근성을 높이기 위한 헤더 버튼



[그림 17] 각 시각화 대시보드로 라우트 가능한 버튼들

접근성을 확보 할 수 있었다.

이후의 개선점

1. 개체수 감지 시스템

현재는 두세마리의 대체 오브젝트를 이용하여 테스트를 진행하였으나, 실제 보호소 환경에서는 수십, 수백 마리의 동물을 동시에 인식해야 하는 상황이 생긴다. 이 경우 현재의 '미달 상태가 감지 될 경우 10초간 대기한다. 이후 미달 상태가 지속 될 경우 DB에 저장 후 경보를 발생시킨다.'라는 방식은 임시방편이 될 가능성이 높다. 때문에 보다 확실하게 각 객체를 인식할 수 있는 방법이 필요할 듯 하다. 부분 라벨링 등의 방식으로 해당 문제를 해결 할 수 있을지 고민해볼 필요가 있다.

2. 이상객체(야생동물) 감지 시스템

현재의 시스템으로는 '이미 등록 되어 있는 야생 동물'에 대한 감지만 이루어지고 있다. 그러나 야생동물들은 다양한 원인으로 서식지를 옮기곤 한다. 해당 지역에서 출몰하지 않던 야생동물이 출몰하기 시작할 경우, 해당 야생동물에 대한 학습 및 등록을 새로 해야 하는 번거로움이 발생한다.

로드킬 방지 스마트 시스템의 경우, 움직이는 물체를 인식할 경우, 해당 물체가 야생동물인지 판단하는 단계를 거친다. 이러한 로직이 해당 시스템에도 적용 가능할지 판단해야 할 필요가 있다.

3. 지도 서비스

현재는 야생 동물이 출몰하면 경보를 발송하는 것으로 끝이 난다. 그러나 보다 근본적인 해결책을 마련하기 위해서는 지도에 출몰한 위치를 기록 할 필요가 있다. 이러한 과정을 통하여 야생 동물의 주요 출몰 지역을 특정할 수 있게 되고, 특정 된 지역에 대하여 물리적인 대처법을 수립할 수 있게 된다.

프로젝트를 마치며

유기동물은 꾸준히 사회적 문제가 되어왔다. 책임 없는 입양과 파양은 이제 그 추세가 줄어가고 있으나, 아예 없어진 것은 아니다. 유기동물을 완전히 없애는 것이 어렵다면, 최소한 그러한 동물들을 위한 복지나, 그들을 보살피는 이들에게 최소한의 여유를 보장해야 한다 생각했다. 이번 프로젝트를 통하여, 어쩌면 직접 만든 시스템으로 그러한 복지와

여유를 제공 할 수 있으리라 생각할 수 있었다. 이후로도 직접적이진 못할지라도, 간접적으로나마 사회적 문제 해결에 도움이 될 시스템을 작성하고자 한다.